

PLACAMAKER

Monitor de Temperatura y Humedad
con ESP32-C3, Sharp LS027B7DH01 y LVGL 9

Guía de aprendizaje detallada

Para personas que están aprendiendo desarrollo embebido

0 Índice

1. Introducción y visión general del proyecto	3
1.1. ¿Qué hace la aplicación?	3
1.2. Hardware utilizado	3
1.3. Mapa de pines	3
2. Estructura del proyecto ESP-IDF	4
2.1. ¿Qué es ESP-IDF?	4
2.2. Árbol de archivos	4
2.3. CMakeLists.txt de proyecto	4
2.4. CMakeLists.txt del componente main	5
2.5. Gestor de componentes: idf_component.yml	5
2.6. sdkconfig.defaults	5
3. Driver de la pantalla Sharp LS027B7DH01	6
3.1. ¿Qué es una Memory LCD?	6
3.2. Protocolo SPI del Sharp	6
3.2.1. Configuración del bus	6
3.2.2. Trama de escritura de líneas	7
3.2.3. Señal EXTCOMIN	7
3.3. Framebuffer	8
3.4. Conversión RGB565 → 1 bit	8
3.5. Corrección de orientación física	9
4. Driver del sensor SHT4x	10
4.1. El sensor Sensirion SHT4x	10
4.2. Protocolo I2C del SHT4x	10
4.2.1. Formato de los 6 bytes de respuesta	11
4.3. Verificación CRC-8	11
4.4. Conversión de valores raw a magnitudes físicas	11
4.5. Detección automática de dirección I2C	12
4.6. Velocidad I2C: 100 kHz en lugar de 400 kHz	13
5. Integración con LVGL 9	13
5.1. ¿Qué es LVGL?	13
5.2. Fuente de tick	13
5.3. Configuración del display	14
5.3.1. ¿Por qué renderizado parcial?	14
5.4. Flush callback	15

5.5. Timer LVGL para actualización del sensor	15
5.6. Construcción de la interfaz gráfica	16
5.7. Fuente Montserrat 28 y sdkconfig	17
6. Arquitectura de tareas FreeRTOS	17
6.1. Diseño de tareas	17
6.2. Bucle principal de LVGL	17
6.3. Retardo de arranque	18
7. Proceso de compilación y flasheo	18
7.1. Herramientas necesarias	18
7.2. Secuencia de comandos	18
8. Problemas encontrados y soluciones	19
8.1. Sin salida por consola: pines UART en conflicto	19
8.2. Dirección I2C incorrecta: CRC fallando	20
8.3. Errores de CRC por pull-ups débiles	20
8.4. Función lv_tick_set_cb() sin efecto	20
8.5. Fuente Montserrat 28 no disponible	20
8.6. Imagen invertida/reflejada	20
9. Flujo completo de una actualización de pantalla	21
10. Resumen de conceptos clave	22

1 Introducción y visión general del proyecto

Este documento explica paso a paso cómo se construyó una aplicación embebida completa que muestra temperatura y humedad en tiempo real sobre una pantalla de tinta electrónica monocroma. El objetivo es que alguien que está aprendiendo pueda entender no solo *qué* se hizo, sino *por qué* se tomó cada decisión.

1.1 ¿Qué hace la aplicación?

El sistema realiza lo siguiente de forma continua:

1. Lee temperatura y humedad del sensor Sensirion SHT4x cada 60 segundos.
2. Muestra los valores en una pantalla Sharp Memory LCD de 400×240 píxeles.
3. Dibuja la interfaz gráfica usando la librería LVGL 9.
4. Gestiona todo con FreeRTOS sobre un microcontrolador ESP32-C3.

1.2 Hardware utilizado

Componente	Descripción
Microcontrolador	ESP32-C3 (RISC-V 32 bits, 160 MHz, WiFi+BT, JTAG integrado)
Pantalla	Sharp LS027B7DH01 (400×240 px, monocroma, Memory LCD)
Sensor	Sensirion SHT4x (temperatura $\pm 0.2^{\circ}\text{C}$, humedad $\pm 1.8\% \text{RH}$, I2C)
Framework	ESP-IDF 5.2 + FreeRTOS
GUI	LVGL 9.5.0

Cuadro 1: Componentes del sistema

1.3 Mapa de pines

El fichero `pinOut.h` centraliza toda la asignación de pines:

Listing 1: `pinOut.h` – Asignación de pines

```

1 #define I2C_SCL      0    // Sensor: reloj I2C
2 #define I2C_SDA      1    // Sensor: datos I2C
3 #define LCD_EXTCON   3    // Pantalla: señal EXTCOMIN
4 #define SPI_CLK      4    // Pantalla: reloj SPI
5 #define SPI_MOSI     5    // Pantalla: datos SPI
6 #define SPI_CS       6    // Pantalla: chip select

```

¿Por qué un fichero separado para los pines?

Centralizar las definiciones evita buscar números mágicos repartidos por el código. Si el hardware cambia, solo hay que modificar un fichero.

2 Estructura del proyecto ESP-IDF

2.1 ¿Qué es ESP-IDF?

ESP-IDF (*Espressif IoT Development Framework*) es el framework oficial para programar chips ESP32. Proporciona:

- Un sistema de construcción basado en CMake y Ninja.
- Drivers de hardware (SPI, I2C, GPIO, timers...).
- FreeRTOS integrado.
- Un gestor de componentes para añadir librerías externas.

2.2 Árbol de ficheros

```

PLACAMAKER/
+-- CMakeLists.txt      <- CMake del proyecto (nivel raíz)
+-- sdkconfig.defaults  <- Configuración por defecto persistente
+-- pinOut.h            <- Asignación global de pines
+-- lv_conf.h           <- Configuración LVGL (parcialmente ignorada)
+-- main/
    +-- CMakeLists.txt  <- CMake del componente principal
    +-- idf_component.yml <- Dependencias externas (LVGL)
    +-- main.c           <- Punto de entrada y lógica de UI
    +-- sharp_lcd.h/.c   <- Driver de la pantalla Sharp
    +-- sht4x.h/.c       <- Driver del sensor SHT4x

```

2.3 CMakeLists.txt de proyecto

Listing 2: CMakeLists.txt raíz

```

1 cmake_minimum_required(VERSION 3.16)
2 include($ENV{IDF_PATH}/tools/cmake/project.cmake)
3 project(placemaker)
4
5 # Añadir el directorio raíz al include path global
6 # (para que main.c pueda hacer #include "pinOut.h")

```

```
7 idf_build_set_property(COMPILER_OPTIONS "-I${CMAKE_SOURCE_DIR}"  
    APPEND)
```

2.4 CMakeLists.txt del componente main

Listing 3: main/CMakeLists.txt

```
1 idf_component_register(  
2     SRCS  
3         "main.c"  
4         "sharp_lcd.c"  
5         "sht4x.c"  
6     INCLUDE_DIRS  
7         "."      # cabeceras locales (sharp_lcd.h, sht4x.h)  
8         ".."     # pinOut.h en la raíz  
9     REQUIRES  
10         driver      # SPI, I2C, GPIO  
11         esp_timer   # timers de alta resolución  
12 )
```

2.5 Gestor de componentes: idf_component.yml

LVGL se descarga automáticamente del repositorio de componentes de Espressif:

Listing 4: main/idf_component.yml

```
1 dependencies:  
2   idf:  
3     version: ">=5.0.0"  
4   lvgl/lvgl:  
5     version: ">=9.0.0,<10.0.0"
```

Ventaja del gestor de componentes:

No hay que copiar el código de LVGL manualmente. ESP-IDF lo descarga, compila e integra automáticamente la primera vez que se ejecuta `idf.py build`.

2.6 sdkconfig.defaults

Este fichero guarda la configuración que se aplica en compilaciones limpias:

Listing 5: sdkconfig.defaults

```
1 CONFIG_IDF_TARGET="esp32c3"  
2 CONFIG_FREERTOS_HZ=1000          # Tick FreeRTOS a 1 kHz
```

```

3 CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG=y    # Consola por USB-JTAG
4 CONFIG_LOG_DEFAULT_LEVEL_INFO=y
5 CONFIG_LV_FONT_MONTERRAT_28=y          # Fuente grande en LVGL

```

¿Por qué CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG?

El ESP32-C3 tiene JTAG/USB integrado en el propio chip. Los pines UART0 predeterminados (GPIO20/21) estaban conectados a los LEDs de la placa, lo que impedía ver los logs por el puerto serie. Cambiando a USB-JTAG, los logs aparecen en `/dev/ttyACM0` sin hardware adicional.

3 Driver de la pantalla Sharp LS027B7DH01

3.1 ¿Qué es una Memory LCD?

La Sharp LS027B7DH01 es una **Memory LCD**: cada píxel tiene un biestable (*flip-flop*) que mantiene su estado sin corriente continua. Esto produce:

- Consumo ultrabajo en reposo (prácticamente cero).
- Imagen visible con retroiluminación apagada.
- Protocolo SPI especial (no estándar MIPI).

La pantalla es monocroma: 1 bit por píxel. 1 = blanco, 0 = negro.

3.2 Protocolo SPI del Sharp

3.2.1 Configuración del bus

Parámetro	Valor
Velocidad máxima	2 MHz
Modo SPI	Modo 0 (CPOL=0, CPHA=0)
Orden de bits	LSB primero (<i>bit 0 se envía primero</i>)
Chip Select (SCS)	Activo en ALTO (invertido respecto a SPI estándar)

Cuadro 2: Parámetros SPI del Sharp LS027B7DH01

¡Atención! SCS activo en ALTO

El SPI estándar usa CS activo en BAJO. El Sharp usa CS activo en ALTO. Por esto se gestiona el pin CS manualmente como GPIO en lugar de dejarlo al controlador SPI. El parámetro `spics_io_num = -1` indica al driver SPI de ESP-IDF que *no* gestione

el CS automáticamente.

3.2.2 Trama de escritura de líneas

Para actualizar N líneas de la pantalla se envía la siguiente trama:

```
[0x01][addr_L1][50 bytes pix][0x00]
[      ][addr_L2][50 bytes pix][0x00]
...
[      ][addr_LN][50 bytes pix][0x00]
[0x00][0x00]
```

Donde:

- 0x01: comando Write Lines (bit M0=1, enviado LSB-first).
- addr: número de línea de 1 a 240 (la línea 1 es la primera).
- **50 bytes de píxeles**: cada fila tiene 400 píxeles / 8 = 50 bytes. El **bit 0** de cada byte corresponde al píxel más a la izquierda.
- 0x00: dummy byte de fin de línea.
- 0x00 0x00: dummy de fin de trama.

¿Por qué LSB primero?

El protocolo del Sharp define que M0 (el bit de comando) es el *primer* bit que llega al display. En SPI estándar se envía el MSB primero, lo que pondría M7 como primer bit. Al activar `SPI_DEVICE_BIT_LSBFIRST`, el ESP32 invierte el orden de bits dentro de cada byte, así M0 llega efectivamente primero.

3.2.3 Señal EXTCOMIN

Las Memory LCD necesitan una señal de inversión de polaridad periódica para prevenir la degradación del cristal líquido. Se genera con un timer:

Listing 6: Timer EXTCOMIN a 2 Hz

```
1 static void extcomin_timer_cb(void *arg)
2 {
3     static int level = 0;
4     level ^= 1;                      // alterna entre 0 y 1
5     gpio_set_level(LCD_EXTCOMIN, level);
6 }
7
```



```

8 // En sharp_lcd_init():
9 esp_timer_start_periodic(extcomin_timer, 250000); // 250 ms -> 2
    Hz

```

El datasheet especifica entre 1 y 20 Hz. Se usa 2 Hz como valor seguro.

3.3 Framebuffer

Para no recalcular toda la pantalla en cada flush parcial, se mantiene un **framebuffer** en RAM que refleja el estado completo de la pantalla:

```

1 static uint8_t framebuffer[LCD_HEIGHT][LCD_ROW_BYTES];
2 // = framebuffer[240][50] = 12.000 bytes

```

Cuando LVGL actualiza una región, se modifica solo la parte afectada del framebuffer y luego se envían esas líneas al display.

3.4 Conversión RGB565 → 1 bit

LVGL trabaja internamente con píxeles en formato RGB565 (16 bits por píxel, 5 bits rojo, 6 bits verde, 5 bits azul). La pantalla solo acepta 1 bit. La conversión usa la fórmula de **luminancia BT.601**:

$$Y = \frac{77 \cdot R_8 + 150 \cdot G_8 + 29 \cdot B_8}{256} \quad (1)$$

donde R_8, G_8, B_8 son los canales expandidos a 8 bits. Si $Y \geq 128$, el píxel es blanco; si no, negro.

Listing 7: Conversión RGB565 a 1 bit

```

1 uint16_t color = buf[(y - y1) * area_w + (x - x1)];
2
3 // Extraer canales de 5/6/5 bits
4 uint8_t r = (color >> 11) & 0x1F; // 5 bits rojos
5 uint8_t g = (color >> 5) & 0x3F; // 6 bits verdes
6 uint8_t b = color & 0x1F; // 5 bits azules
7
8 // Expandir a 8 bits
9 uint8_t r8 = (r << 3) | (r >> 2);
10 uint8_t g8 = (g << 2) | (g >> 4);
11 uint8_t b8 = (b << 3) | (b >> 2);
12
13 // Luminancia BT.601
14 uint8_t lum = (77u * r8 + 150u * g8 + 29u * b8) >> 8;
15

```

```
16 // Umbral
17 if (lum >= 128) pixel = blanco; else pixel = negro;
```

¿Por qué esos coeficientes (77, 150, 29)?

El ojo humano no percibe todos los colores con la misma intensidad. Es más sensible al verde ($150/255 \approx 59\%$) que al rojo ($77/255 \approx 30\%$) y mucho más que al azul ($29/255 \approx 11\%$). Esta fórmula estándar (BT.601) produce la mejor representación monocroma de una imagen en color.

3.5 Corrección de orientación física

La pantalla estaba montada físicamente rotada 180° , lo que provocaba que la imagen apareciera con ambos ejes invertidos. La solución está en el *flush callback*: en lugar de escribir el píxel LVGL (x, y) en la posición física (x, y) del framebuffer, se calcula su posición física real:

Listing 8: Corrección de orientación en sharp_lcd_flush()

```
1 for (int32_t y = y1; y <= y2; y++) {
2     // Invertir Y: la línea LVGL 0 va a la fila física 239
3     int32_t phy_y = (LCD_HEIGHT - 1) - y;
4
5     for (int32_t x = x1; x <= x2; x++) {
6         // ... conversion RGB565 -> lum ...
7
8         // Invertir X: el píxel LVGL 0 va a la columna física 399
9         int32_t phy_x = (LCD_WIDTH - 1) - x;
10        int byte_idx  = phy_x / 8;
11        int bit_idx   = phy_x % 8;
12
13        framebuffer[phy_y][byte_idx] |= (1u << bit_idx); //
14        blanco
15    }
16 }
17 // Enviar el rango físico afectado (siempre en orden creciente)
18 int32_t phy_y1 = (LCD_HEIGHT - 1) - y2;
19 int32_t phy_y2 = (LCD_HEIGHT - 1) - y1;
20 lcd_send_lines(phy_y1, phy_y2);
```

¿Por qué no usar `lv_display_set_rotation(ROTATION_180)`?

LVGL tiene una función de rotación 180°, pero esta invierte tanto X como Y. Si la pantalla solo necesitara inversión en Y, funcionaría en combinación con corrección de X en el driver. Sin embargo, al ser la orientación física una propiedad del hardware (no de la lógica), es más limpio mantener la corrección en el driver y dejar LVGL en `ROTATION_0` (sin rotación).

¿Por qué `lcd_send_lines` recibe el rango físico?

El protocolo Sharp requiere que las direcciones de línea se envíen en orden *creciente*. Si se invirtiera el orden de las direcciones en la trama SPI, el display interpretaría los datos de forma incorrecta, causando artefactos visuales en el eje X. Al calcular `phy_y1` y `phy_y2` antes de llamar a `lcd_send_lines`, las líneas siempre se envían de menor a mayor dirección.

4 Driver del sensor SHT4x

4.1 El sensor Sensirion SHT4x

El SHT4x es un sensor digital de temperatura y humedad que se comunica por I2C. Existe en varias variantes con distintas precisiones y direcciones I2C:

Variante	Dirección I2C	Precisión
SHT40-xD1B	0x44, 0x45 o 0x46	$\pm 1.8\% \text{RH}$, $\pm 0.2^\circ \text{C}$
SHT41-AD1B	0x44	$\pm 1.8\% \text{RH}$, $\pm 0.2^\circ \text{C}$
SHT45-AD1B	0x44	$\pm 1.0\% \text{RH}$, $\pm 0.1^\circ \text{C}$

Cuadro 3: Variantes del SHT4x

4.2 Protocolo I2C del SHT4x

La comunicación sigue siempre el mismo patrón con *dos transacciones I2C separadas* (no *repeated start*):

1. **Escritura:** enviar la dirección del sensor + comando de medición (`0xFD` = medición de alta precisión).
2. **Espera:** el sensor necesita hasta 8.3 ms para completar la medición. Se espera 15 ms para tener margen.
3. **Lectura:** leer 6 bytes de resultado.

4.2.1 Formato de los 6 bytes de respuesta

Byte 0: T_MSB (byte alto de temperatura)
Byte 1: T_LSB (byte bajo de temperatura)
Byte 2: T_CRC (checksum CRC-8 de los bytes 0 y 1)
Byte 3: RH_MSB (byte alto de humedad)
Byte 4: RH_LSB (byte bajo de humedad)
Byte 5: RH_CRC (checksum CRC-8 de los bytes 3 y 4)

4.3 Verificación CRC-8

Cada par de bytes va seguido de un checksum para detectar errores de transmisión. El algoritmo es CRC-8 con polinomio 0x31 y semilla 0xFF:

Listing 9: Implementación del CRC-8 Sensirion

```
1 static uint8_t sht4x_crc8(const uint8_t *data, size_t len)
2 {
3     uint8_t crc = 0xFF;           // semilla inicial
4     for (size_t i = 0; i < len; i++) {
5         crc ^= data[i];           // XOR con el byte de datos
6         for (int b = 0; b < 8; b++) {
7             // si el bit mas significativo es 1:
8             if (crc & 0x80)
9                 crc = (crc << 1) ^ 0x31; // desplazar y XOR con
                polinomio
10            else
11                crc = (crc << 1);         // solo desplazar
12        }
13    }
14    return crc;
15 }
```

¿Por qué CRC?

I2C puede sufrir interferencias electromagnéticas, especialmente a 400 kHz con cables largos o near de fuentes de ruido. Sin CRC, un error de bit produciría lecturas incorrectas de temperatura sin que el software lo detectara. Con CRC, cualquier byte corrupto se detecta y se descarta la lectura.

4.4 Conversión de valores raw a magnitudes físicas

El sensor devuelve valores enteros de 16 bits que se convierten con las fórmulas del datasheet:

$$T [^{\circ}\text{C}] = -45 + 175 \cdot \frac{T_{\text{raw}}}{65535} \quad (2)$$

$$\text{RH} [\%] = \text{clamp}\left(-6 + 125 \cdot \frac{\text{RH}_{\text{raw}}}{65535}, 0, 100\right) \quad (3)$$

Listing 10: Conversión de raw a °C y %RH

```

1 uint16_t t_raw = (uint16_t)(data[0] << 8) | data[1];
2 uint16_t rh_raw = (uint16_t)(data[3] << 8) | data[4];
3
4 float t = -45.0f + 175.0f * (float)t_raw / 65535.0f;
5 float rh = -6.0f + 125.0f * (float)rh_raw / 65535.0f;
6
7 // Limitar humedad al rango fisico [0, 100]
8 if (rh < 0.0f) rh = 0.0f;
9 if (rh > 100.0f) rh = 100.0f;

```

4.5 Detección automática de dirección I2C

Un problema frecuente al trabajar con sensores I2C es no saber con certeza qué dirección usa el dispositivo en la placa concreta. Para resolverlo, `sht4x_init()` prueba las tres direcciones posibles:

Listing 11: Escaneo de direcciones I2C

```

1 static const uint8_t SHT4X_ADDRS[] = { 0x44, 0x45, 0x46 };
2
3 uint8_t found_addr = 0;
4 for (int i = 0; i < 3; i++) {
5     // i2c_master_probe() envía solo el byte de direccion y
6     // espera ACK
7     esp_err_t probe = i2c_master_probe(i2c_bus, SHT4X_ADDRS[i],
8     50);
9     ESP_LOGI(TAG, "Probe 0x%02X -> %s",
10     SHT4X_ADDRS[i], probe == ESP_OK ? "ACK" : "NACK");
11     if (probe == ESP_OK && found_addr == 0)
12         found_addr = SHT4X_ADDRS[i];
13 }

```

En este proyecto el sensor respondió en **0x45** (no en 0x44 como asumíamos), lo que hubiera sido imposible de detectar sin el escaneo.

4.6 Velocidad I2C: 100 kHz en lugar de 400 kHz

Inicialmente se configuró el bus a 400 kHz (*fast mode*). Esto producía errores de CRC en cada lectura. La causa es que las resistencias de pull-up *internas* del ESP32-C3 son de aproximadamente 45 k Ω , muy superiores a los 10 k Ω que recomienda el datasheet del SHT4x para 400 kHz. Con pull-ups tan débiles, el tiempo de subida de la señal es demasiado largo y los bits se corrompen.

Velocidad	Tiempo de subida máx.	Pull-up recomendado
100 kHz (Standard)	1000 ns	$\leq 10\text{ k}\Omega$
400 kHz (Fast)	300 ns	$\leq 1\text{ k}\Omega$

Cuadro 4: Requisitos de pull-up según velocidad I2C

Reduciendo a 100 kHz, los 45 k Ω internos son suficientes para tiempos de subida dentro de especificación. El sensor funciona correctamente.

5 Integración con LVGL 9

5.1 ¿Qué es LVGL?

LVGL (*Light and Versatile Graphics Library*) es una librería gráfica de código abierto diseñada para microcontroladores. Proporciona:

- Widgets (botones, etiquetas, barras...).
- Sistema de layouts y estilos CSS-like.
- Soporte para distintos formatos de color y profundidades de bit.
- Renderizado parcial para dispositivos con poca RAM.

5.2 Fuente de tick

LVGL necesita saber cuánto tiempo ha transcurrido para gestionar animaciones, timers y eventos. En LVGL 9.5 esto se configura con `lv_tick_set_cb()`:

Listing 12: Fuente de tick para LVGL 9.5

```

1 static uint32_t lvgl_tick_cb(void)
2 {
3     // esp_timer_get_time() devuelve microsegundos
4     // LVGL necesita milisegundos
5     return (uint32_t)(esp_timer_get_time() / 1000LL);
6 }
7
```

```
8 // En lvgl_task():
9 lv_init();
10 lv_tick_set_cb(lvgl_tick_cb);
```

Cambio en LVGL 9.5: LV_TICK_CUSTOM eliminado

En LVGL 8.x y versiones anteriores de 9.x existía la macro `LV_TICK_CUSTOM` en `lv_conf.h` para configurar la fuente de tick. En LVGL 9.5 esta macro fue eliminada. Si se intenta usarla, el tick nunca avanza y los timers no funcionan. La forma correcta desde 9.5 es llamar a `lv_tick_set_cb()` en código.

5.3 Configuración del display

Listing 13: Creación y configuración del display LVGL

```
1 // Buffers de renderizado parcial: 20 filas x 400 pixeles x 2
  bytes
2 #define LVGL_BUFFER_ROWS 20
3 #define LVGL_BUF_BYTES    (LCD_WIDTH * LVGL_BUFFER_ROWS * 2)
4
5 static uint8_t lvgl_buf1[LVGL_BUF_BYTES]; // buf. primario
6 static uint8_t lvgl_buf2[LVGL_BUF_BYTES]; // buf. secundario
7
8 // Crear display de 400x240
9 lv_display_t *disp = lv_display_create(LCD_WIDTH, LCD_HEIGHT);
10
11 // Registrar el callback que envia pixels a la pantalla
12 lv_display_set_flush_cb(disp, flush_cb);
13
14 // Asignar los buffers en modo parcial
15 lv_display_set_buffers(disp, lvgl_buf1, lvgl_buf2,
16                       LVGL_BUF_BYTES,
17                       LV_DISPLAY_RENDER_MODE_PARTIAL);
```

5.3.1 ¿Por qué renderizado parcial?

Un framebuffer completo RGB565 para 400×240 px necesitaría $400 \times 240 \times 2 = 192,000$ bytes de RAM. El ESP32-C3 solo tiene 320 KB de SRAM total. Con renderizado parcial, LVGL solo necesita dos buffers de 20 filas: $400 \times 20 \times 2 \times 2 = 32,000$ bytes. LVGL renderiza la pantalla en bloques y llama al flush callback repetidamente.

5.4 Flush callback

El flush callback es el puente entre LVGL y el driver de la pantalla:

Listing 14: Flush callback LVGL ->Sharp LCD

```
1 static void flush_cb(lv_display_t *disp,
2                     const lv_area_t *area,
3                     uint8_t *px_map)
4 {
5     // Pasar el area y los pixeles al driver del Sharp
6     sharp_lcd_flush(area->x1, area->y1,
7                     area->x2, area->y2,
8                     px_map);
9
10    // Notificar a LVGL que el flush ha terminado
11    lv_display_flush_ready(disp);
12 }
```

¡lv_display_flush_ready() es obligatorio!

LVGL espera esta llamada para saber que puede reutilizar el buffer de renderizado. Si se omite, LVGL se bloquea indefinidamente. Puede llamarse desde dentro del flush callback (si el envío es síncrono, como en este caso) o desde una interrupción DMA (si es asíncrono).

5.5 Timer LVGL para actualización del sensor

LVGL tiene su propio sistema de timers basado en el tick. Se usa para refrescar los datos del sensor cada 60 segundos sin crear una tarea FreeRTOS adicional:

Listing 15: Timer LVGL de 60 segundos

```
1 static void sensor_update_cb(lv_timer_t *timer)
2 {
3     float temp = 0.0f, hum = 0.0f;
4     char buf[48];
5
6     if (sht4x_read(&temp, &hum) == ESP_OK) {
7         snprintf(buf, sizeof(buf),
8                 "Temp:\n%.1f \xc2\xbb0"C\n\nHum:\n%.1f%%",
9                 temp, hum);
10    } else {
11        snprintf(buf, sizeof(buf),
12                "Temp:\n-- \xc2\xbb0"C\n\nHum:\n--.-%%");
```



```
13     }
14     lv_label_set_text(sensor_label, buf);
15 }
16
17 // Llamada inmediata al arrancar + timer de 60 s
18 sensor_update_cb(NULL);
19 lv_timer_create(sensor_update_cb, 60000, NULL);
```

Codificación del símbolo de grado (°)

El carácter ° en UTF-8 se codifica como dos bytes: 0xC2 0xB0. En el código se escribe como "\xc2\xb0" porque el compilador podría interpretar el carácter directamente de forma errónea dependiendo del locale. Con los bytes explícitos el comportamiento es siempre correcto.

5.6 Construcción de la interfaz gráfica

Listing 16: Creación de la UI: fondo blanco + cuadrado + etiqueta

```
1 // Fondo blanco
2 lv_obj_t *scr = lv_screen_active();
3 lv_obj_set_style_bg_color(scr, lv_color_white(), LV_PART_MAIN);
4 lv_obj_set_style_bg_opa(scr, LV_OPA_COVER, LV_PART_MAIN);
5
6 // Cuadrado centrado de 200x200 px con borde negro
7 lv_obj_t *box = lv_obj_create(scr);
8 lv_obj_set_size(box, 200, 200);
9 lv_obj_align(box, LV_ALIGN_CENTER, 0, 0);
10 lv_obj_set_style_bg_color(box, lv_color_white(), LV_PART_MAIN);
11 lv_obj_set_style_border_color(box, lv_color_black(),
12     LV_PART_MAIN);
13 lv_obj_set_style_border_width(box, 3, LV_PART_MAIN);
14 lv_obj_set_style_radius(box, 0, LV_PART_MAIN); // sin bordes
15     redondeados
16
17 lv_obj_set_style_pad_all(box, 10, LV_PART_MAIN);
18 lv_obj_clear_flag(box, LV_OBJ_FLAG_SCROLLABLE); // sin scroll
19
20 // Etiqueta de temperatura y humedad
21 sensor_label = lv_label_create(box);
22 lv_label_set_long_mode(sensor_label, LV_LABEL_LONG_WRAP);
23 lv_obj_set_width(sensor_label, lv_pct(100));
24 lv_obj_set_style_text_font(sensor_label,
25     &lv_font_montserrat_28, LV_PART_MAIN);
```

```

23 lv_obj_set_style_text_color(sensor_label, lv_color_black(),
    LV_PART_MAIN);
24 lv_obj_set_style_text_align(sensor_label, LV_TEXT_ALIGN_CENTER,
    LV_PART_MAIN);
25 lv_obj_align(sensor_label, LV_ALIGN_CENTER, 0, 0);

```

5.7 Fuente Montserrat 28 y sdkconfig

LVGL compilado con ESP-IDF usa `sdkconfig` como única fuente de configuración. El fichero `lv_conf.h` existe pero es **ignorado** porque la variable `CONFIG_LV_CONF_SKIP=y` está activa. Por eso, para habilitar la fuente Montserrat de 28 puntos hay que hacerlo *en `sdkconfig`*, no en `lv_conf.h`:

Listing 17: En `sdkconfig.defaults`

```

1 CONFIG_LV_FONT_MONTSEERRAT_28=y

```

Error común: editar `lv_conf.h` no tiene efecto

Si `CONFIG_LV_CONF_SKIP=y` está activado (es el valor por defecto en la integración con ESP-IDF), cualquier cambio en `lv_conf.h` se ignora completamente. Todos los ajustes de LVGL deben hacerse mediante `idf.py menuconfig` o editando `sdkconfig.defaults`.

6 Arquitectura de tareas FreeRTOS

6.1 Diseño de tareas

Tarea	Función	Stack	Prioridad
lvgl	Todo el renderizado LVGL, lectura sensor y actualización UI	8192 B	5
app_main	Heartbeat cada 30 s (monitorización)	por defecto	1

Cuadro 5: Tareas FreeRTOS del sistema

6.2 Bucle principal de LVGL

Listing 18: Bucle principal de la tarea LVGL

```

1 while (1) {
2     // lv_timer_handler() procesa todos los timers y
    renderizados pendientes.
3     // Devuelve el tiempo en ms hasta el proximo evento.

```

```
4     uint32_t delay_ms = lv_timer_handler();
5
6     // Limitar: si no hay nada pendiente, esperar maximo 10 ms
7     // para que el bucle no corra demasiado rapido ni demasiado
8     // lento
9     if (delay_ms == 0 || delay_ms > 100) delay_ms = 10;
10
11     vTaskDelay(pdMS_TO_TICKS(delay_ms));
12 }
```

¿Por qué vTaskDelay en lugar de un bucle activo?

FreeRTOS es un sistema multitarea cooperativo/preemptivo. Si la tarea LVGL nunca cede la CPU (sin vTaskDelay), otras tareas de igual o menor prioridad nunca ejecutan. vTaskDelay pone la tarea en estado “bloqueada” durante el tiempo indicado, cediendo la CPU al scheduler.

6.3 Retardo de arranque

```
1 void app_main(void)
2 {
3     vTaskDelay(pdMS_TO_TICKS(4000)); // 4 segundos
4     // ...
5 }
```

Este retardo existe porque el ESP32-C3 usa USB-JTAG como consola. Cuando el chip arranca, el host (PC) necesita detectar el dispositivo USB y abrir el puerto serie. Si se imprimen logs antes de que el puerto esté listo, los primeros mensajes se pierden. Con 4 segundos de margen, el PC siempre tiene tiempo de enumerar el dispositivo USB.

7 Proceso de compilación y flasheo

7.1 Herramientas necesarias

1. **ESP-IDF 5.2**: framework descargado en ~/ESP32Y/esp-idf/
2. **Toolchain RISC-V**: compilador riscv32-esp-elf-gcc, instalado en ~/.espressif/tools/riscv32-esp-elf-gcc
3. **Python 3**: para los scripts de ESP-IDF y esptool.

7.2 Secuencia de comandos

Listing 19: Compilar, flashear y monitorizar

```
1 # 1. Activar el entorno ESP-IDF
2 source ~/ESP32Y/esp-idf/export.sh
3
4 # 2. Configurar el target (primera vez o tras borrar build/)
5 idf.py set-target esp32c3
6
7 # 3. (Opcional) Configuración interactiva
8 idf.py menuconfig
9
10 # 4. Compilar
11 idf.py build
12
13 # 5. Flashear por USB-JTAG
14 idf.py -p /dev/ttyACM0 flash
15
16 # 6. Ver logs en tiempo real
17 idf.py -p /dev/ttyACM0 monitor
```

8 Problemas encontrados y soluciones

Este capítulo documenta los problemas reales que surgieron durante el desarrollo y cómo se resolvieron. Aprender de los errores es tan valioso como aprender del código correcto.

8.1 Sin salida por consola: pines UART en conflicto

Síntoma: el chip flasheaba correctamente pero no aparecía ningún log por el puerto serie.

Causa: ESP-IDF configura por defecto la consola en UART0, cuyos pines son GPIO20 y GPIO21. En esta placa, esos pines están conectados a los LEDs (rojo y amarillo), no a ningún conversor USB-UART.

Solución: cambiar la consola a USB-JTAG (integrado en el ESP32-C3):

Listing 20: En sdkconfig

```
1 CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG=y
2 CONFIG_ESP_CONSOLE_SECONDARY_NONE=y
```

8.2 Dirección I2C incorrecta: CRC fallando

Síntoma: el log mostraba `E sht4x: CRC de temperatura inválido` en cada lectura. La comunicación I2C funcionaba (no había error de timeout).

Causa: se asumió que el sensor tenía la dirección `0x44` (la más común), pero la variante soldada en la placa usaba la dirección **`0x45`**. Un dispositivo diferente respondía en `0x44` devolviendo datos sin sentido.

Solución: añadir un escaneo automático de las tres direcciones posibles (`0x44`, `0x45`, `0x46`) durante la inicialización.

8.3 Errores de CRC por pull-ups débiles

Síntoma: incluso con la dirección correcta, aparecían errores de CRC esporádicos a 400 kHz.

Causa: las resistencias de pull-up internas del ESP32-C3 (45 k Ω) son demasiado débiles para I2C a 400 kHz. La señal no sube lo suficientemente rápido y los bits se corrompen.

Solución: reducir la velocidad I2C a 100 kHz.

8.4 Función `lv_tick_set_cb()` sin efecto

Síntoma: los timers de LVGL nunca disparaban. El sensor se leía una vez al arrancar pero nunca más.

Causa: se intentó configurar el tick mediante `LV_TICK_CUSTOM` en `lv_conf.h`, que fue eliminado en LVGL 9.5.

Solución: llamar a `lv_tick_set_cb(lvgl_tick_cb)` en código después de `lv_init()`.

8.5 Fuente Montserrat 28 no disponible

Síntoma: error de compilación: símbolo `lv_font_montserrat_28` no declarado.

Causa: se había activado la fuente en `lv_conf.h` pero `CONFIG_LV_CONF_SKIP=y` hace que LVGL ignore ese fichero.

Solución: añadir `CONFIG_LV_FONT_MONTSEERRAT_28=y` al `sdkconfig.defaults`.

8.6 Imagen invertida/reflejada

Síntoma: la imagen aparecía con ambos ejes invertidos (rotada 180°).

Causa: la pantalla estaba montada físicamente rotada 180° en la PCB.

Solución: corregir ambos ejes en el flush callback calculando las posiciones físicas

reales:

$$\text{phy_y} = (\text{LCD_HEIGHT} - 1) - y$$

$$\text{phy_x} = (\text{LCD_WIDTH} - 1) - x$$

9 Flujo completo de una actualización de pantalla

Para consolidar el aprendizaje, aquí se describe paso a paso qué ocurre cada vez que el timer de 60 segundos dispara:

1. El timer LVGL llama a `sensor_update_cb()`.
2. La función llama a `sht4x_read(&temp, &hum)`.
3. Dentro de `sht4x_read()`:
 - a) Se envía el comando 0xFD al sensor por I2C.
 - b) Se espera 15 ms.
 - c) Se leen 6 bytes.
 - d) Se verifican los dos CRC.
 - e) Se aplican las fórmulas de conversión.
4. Se formatea el texto y se llama a `lv_label_set_text()`.
5. LVGL marca la etiqueta como “sucia” (*dirty*).
6. En la próxima iteración del bucle LVGL, `lv_timer_handler()` detecta regiones sucias y las renderiza en el buffer parcial.
7. LVGL llama a `flush_cb()` con el área renderizada.
8. `flush_cb()` llama a `sharp_lcd_flush()`.
9. Dentro de `sharp_lcd_flush()`:
 - a) Se convierte cada píxel RGB565 a 1 bit (luminancia BT.601).
 - b) Se aplica la corrección de orientación (`phy_x`, `phy_y`).
 - c) Se actualiza el framebuffer.
 - d) Se llama a `lcd_send_lines()` con el rango físico.
10. Dentro de `lcd_send_lines()`:
 - a) Se construye la trama SPI en `tx_buf`.

- b) Se pone SCS en ALTO.
 - c) Se envía la trama por SPI (DMA).
 - d) Se pone SCS en BAJO.
11. Se llama a `lv_display_flush_ready()` para liberar el buffer.
 12. La pantalla actualiza físicamente los píxeles afectados.

10 Resumen de conceptos clave

Concepto	Qué aprender de este proyecto
SPI LSB-first	Algunos dispositivos requieren el bit menos significativo primero; hay que configurarlo explícitamente en el driver.
Memory LCD	Las pantallas de tinta electrónica mantienen la imagen sin corriente; el protocolo es propietario y requiere EXTCOMIN.
I2C scanning	Nunca asumir la dirección I2C; siempre escanear o comprobar el datasheet de la variante exacta.
CRC	Los checksums protegen los datos de errores de transmisión en entornos ruidosos.
LVGL + ESP-IDF	La configuración de LVGL va en <code>sdkconfig</code> , no en <code>lv_conf.h</code> , cuando <code>CONFIG_LV_CONF_SKIP=y</code> .
Flush callback	El puente entre la librería gráfica y el driver de pantalla; <code>lv_display_flush_ready()</code> es obligatorio.
Framebuffer	Mantener una copia del estado de la pantalla en RAM permite actualizaciones parciales eficientes.
FreeRTOS	<code>vTaskDelay</code> cede la CPU; sin él, otras tareas nunca ejecutan.
Pull-ups I2C	A mayor velocidad I2C, menor resistencia de pull-up necesaria. Los pull-ups internos suelen ser demasiado débiles para 400 kHz.

Cuadro 6: Conceptos clave del proyecto